

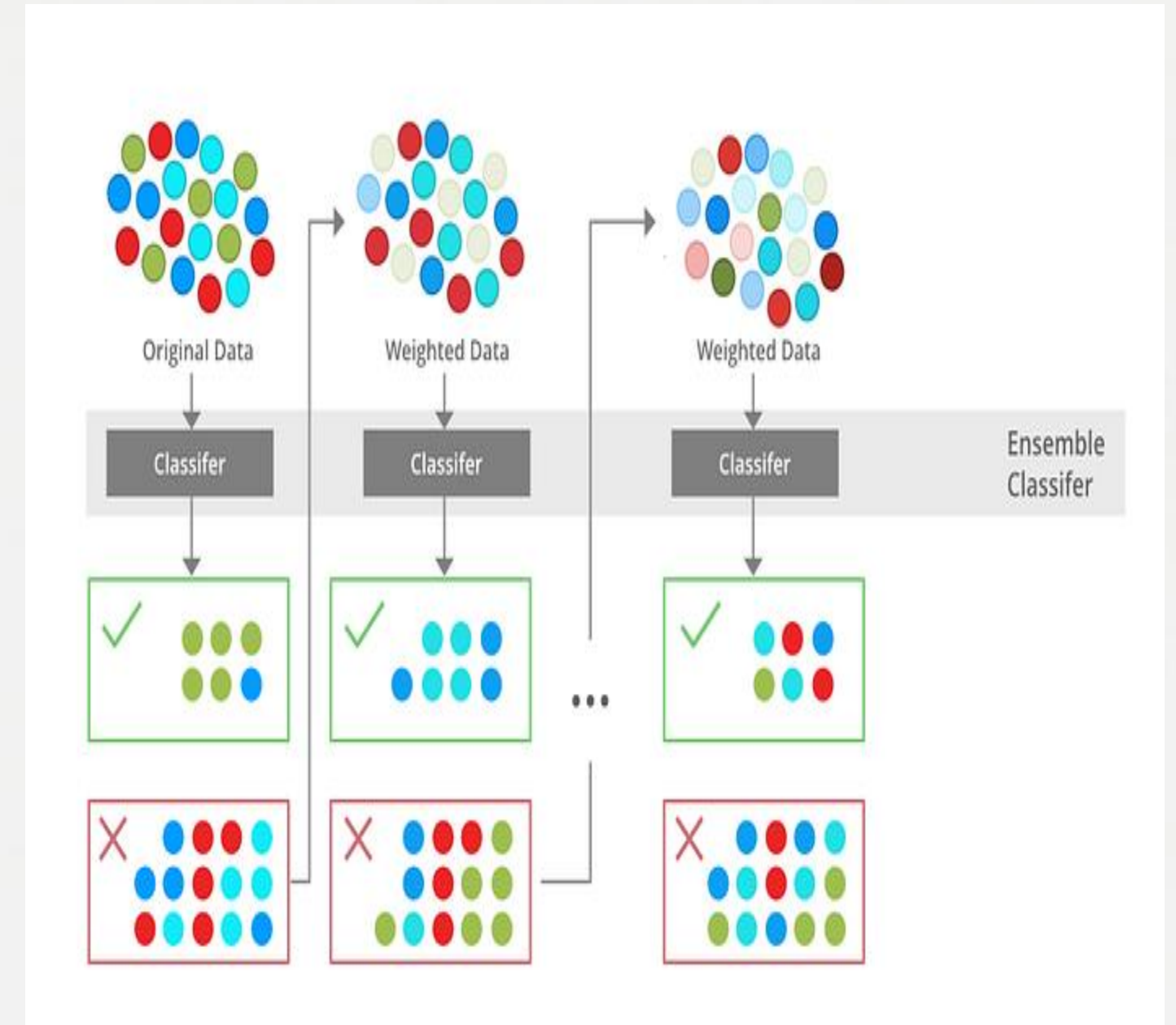
XGBoost

Extreme Gradient Boosting

Anuraj Sapkota

Introduction

- Based on Gradient Boosted Decision Trees (GBDT).
- Popular for speed, scalability and accuracy.
- Widely used in Kaggle, recommendation system, predictive analytics.



How does it works ?

- It works by combining a number of weak learners to form a strong learner.
- XGBoost works by training a number of decision trees. Each tree trained on a subset of data and prediction from each tree are combined to form final prediction.

— How does it works ?

1. Base model and its initial prediction.
2. Compute residuals or errors.
3. Fit a new decision tree to residuals.
4. Add tree to model with learning rate.
5. Repeat until low bias and low variance.

— Hyperparameters

- max_depth: maximum depth of decision trees.
- gamma: minimum loss reduction to make a split.
- subsample: fraction of training data used to train each tree.
- n_estimators: number of trees.
- Colsample_bytree: features per tree

Key Features

- Parallel processing
- Optimized Data Structures
- Cache Awareness
- Out of core computing
- Distributed computing
- Regularization

1. Parallel Processing

- Split dataset and distributed across multiple cores.
- Uses histogram-based algorithm.
- Each subset calculates the quantiles in parallel and the statistic is pulled together to form an approximate histogram of quantiles.

2. Optimized Data Structure

- XGBoost divides the dataset into multiple blocks in a Compressed Sparse Column format (CSC) to distribute the blocks to multiple cores for parallel learning.

Header 1	Header 2	Header 3	Header 4
	BLOCK 1		
	BLOCK 2		
	BLOCK 3		
	BLOCK 4		

3. Cache Awareness

- Since, cache memory is fastest to access it stores first and second order derivatives within XGBoost to calculate the scores for each node and leaf in the tree.
- Improves training speeds in large dataset.

4. Out of core Computing.

- Trains data larger than memory.
- Splits huge dataset into chunks and process each blocks sequentially.

5. Distributed Computing.

- Multi-machine training.
- Supports fully distributed GPU training using Dask, Spark and PySpark.
- Have one centralized scheduler, one or more workers and one or more clients as entrypoint for submitting tasks to the cluster.

6. Regularization and Pruning.

- Penalizing complex models to reduce overfitting.
- Uses L1: add absolute penalty encourages sparsity.
- L2: adds squared penalty smooth leaf weights.

— Feature Importance

- Measurements of how much each features contribute the XGBoost model.
- Weight: how many times a particular feature is used for splitting across the trees. Higher counts– more important feature
- Gain: how much a feature reduces the error when splitting.
- Cover: affected samples number by a feature when used in splits. Higher coverage – feature affects more data points

— SHAP Value

- Shapely Additive exPlanations.
- How much each features contributes to a single prediction.
- Positive SHAP value: features contributes in higher prediction.
- Negative SHAP value: feature contributes in lower prediction.
- Gives both local and global interpretability.

The End

THANK YOU FOR LISTENING